Отчёт по индивидуальному заданию № 3

Архитектура компьютера и операционные системы

Вариант 9. Магазин с несколькими отделами

ФИО: Жиляков Роман Евгеньевич

Группа: БПИ252

1. Условие задания

В магазине работают три отдела, в каждом находится один продавец. Покупатели приходят через заданный интервал и посещают от одного до трёх различных отделов по заранее определённому маршруту.

Если продавец занят, покупатель становится в очередь FIFO. После выполнения маршрута он покидает магазин.

При наступлении времени CLOSE вход закрывается. Покупатели, успевшие войти, продолжают обслуживание.

Необходимо реализовать модель магазина на языке C для Linux с использованием взаимодействующих процессов и средств POSIX.

2. Описание модели

В моделировании участвуют управляющий процесс, три процесса продавцов и N процессов покупателей. Всего используется N + 4 процесса.

Управляющий процесс отвечает за модельное время, очереди, обработку событий и журнал. Продавцы обслуживают покупателей, а покупатели последовательно выполняют свои маршруты.

Реализованы четыре режима взаимодействия процессов:

Режим	**Способ взаимодействия**
fork-named	Дочерние процессы, разделяемая память и именованные POSIX-семафоры
fork-unnamed	Дочерние процессы, разделяемая память и неименованные POSIX-семафоры
shm	Независимые процессы, разделяемая память и именованные семафоры
mq	Независимые процессы, очереди сообщений и именованные семафоры

Во всех режимах используются одинаковые правила моделирования магазина.

3. Алгоритм работы

Сначала программа проверяет входные данные и создаёт необходимые объекты взаимодействия.

В режимах fork-named и fork-unnamed процессы создаются через fork(). В режимах shm и mq участники запускаются независимо и подключаются к сессии управляющего процесса.

Первый покупатель приходит в момент 0. Время появления покупателя i определяется формулой:

Время появления = (i - 1) × INTERVAL

Основные этапы моделирования:

1. Проверка времени закрытия магазина.
2. Завершение текущих обслуживаний.
3. Переход покупателей между отделами.
4. Появление новых покупателей.
5. Начало обслуживания в свободных отделах.
6. Проверка условия завершения.

Очереди работают по правилу FIFO. Каждый продавец обслуживает только одного покупателя одновременно.

Продолжительность обслуживания выбирается в заданном диапазоне с помощью rand_r(). Для согласования процессов используются POSIX-семафоры и подтверждения выполненных команд.

4. Состав проекта

Основные файлы находятся в папке hw3.

Файл	**Назначение**
main.c	Исходный код модели и четырёх режимов IPC
Makefile	Сборка, тестирование и очистка
report.pdf	Итоговый отчёт

При сборке создаются исполняемые файлы shop и actor. Первый управляет магазином, второй используется для независимого запуска участников.

5. Интерфейс и формат входных данных

Запуск программы:

./shop MODE INPUT_FILE LOG_FILE DELAY_MS SEED

Аргументы:

Аргумент	**Назначение**
MODE	Режим взаимодействия процессов
INPUT_FILE	Путь к входному файлу
LOG_FILE	Путь к журналу
DELAY_MS	Задержка отображения в миллисекундах
SEED	Начальное значение генератора случайных чисел

Формат входного файла:

N INTERVAL CLOSE

MIN1 MAX1

MIN2 MAX2

MIN3 MAX3

K D1 ... DK

После параметров отделов указываются N маршрутов покупателей.

Допустимые значения: N от 0 до 100, INTERVAL и CLOSE от 0 до 100000, время обслуживания от 1 до 10000, длина маршрута от 1 до 3. Номера отделов не должны повторяться в одном маршруте.

6. Сборка и запуск

Программа рассчитана на Linux и компилируется с использованием GCC и POSIX API.

Основные команды:

make -C hw3

make -C hw3 test

make -C hw3 clean

Пример запуска из папки hw3:

./shop fork-named ../hw2/tests/normal.txt /tmp/hw3.log 0 42

В режимах shm и mq управляющий процесс запускается первым. Затем отдельно запускаются три продавца и N покупателей с помощью программы actor.

Формат запуска участника:

./actor MODE SESSION seller|buyer NUMBER

SESSION соответствует идентификатору управляющего процесса.

7. Ввод-вывод и журнал

Входной файл читается через системные вызовы open() и read(). Числовые параметры проверяются с помощью strtol().

Программа проверяет допустимые диапазоны, маршруты покупателей, повреждённые и лишние данные. Предусмотрена защита входного файла от перезаписи журналом.

В терминале и журнале отображаются вход покупателей, маршруты, переходы, очереди, начало и завершение обслуживания, закрытие магазина и итоговая статистика.

Каждое событие сопровождается модельным временем в формате [t=...].

8. Завершение программы

Обычное завершение наступает после закрытия магазина и выполнения маршрутов всеми вошедшими покупателями.

Программа поддерживает завершение по сигналам SIGINT и SIGTERM. При прерывании сохраняется итоговая статистика, завершаются процессы и освобождаются ресурсы IPC.

В зависимости от режима используются sem_close(), sem_unlink(), sem_destroy(), munmap(), shm_unlink(), mq_close() и mq_unlink().

Коды завершения:

Код Значение

0	Успешное завершение
1	Ошибка выполнения
130	Прерывание SIGINT
143	Прерывание SIGTERM

Проверяемые соотношения:

Вошли = вышли + внутри

Всего = вошли + не допущены + ещё не пришли

9. Тестирование и результаты

Для автоматической проверки используется команда make test.

Тестирование включает 20 запусков: по пять сценариев для каждого из четырёх режимов.

Сценарий	**Проверяемое поведение**
normal	Обычная работа магазина
empty	Отсутствие покупателей
closing	Закрытие входа
stress	100 покупателей и 300 покупок
invalid	Некорректные входные данные

Все автоматические проверки завершились успешно.

В нагрузочном сценарии каждый из 100 покупателей посещает три отдела. Всего выполняется 300 покупок, каждый продавец обслуживает 100 покупателей.

Дополнительно проверены очереди FIFO, итоговая статистика, обработка SIGINT и SIGTERM, параллельные запуски магазинов и удаление IPC-объектов.

Программа успешно собирается компиляторами GCC и Clang без предупреждений.

10. Ограничения модели

Количество отделов фиксировано и равно трём. Число покупателей ограничено 100. Маршруты задаются заранее, а время обслуживания измеряется целыми модельными тактами.

В независимых режимах необходимо запустить всех участников. При принудительном завершении через SIGKILL штатная очистка ресурсов не гарантируется.

11. Использование ИИ при тестировании и оформлении работы

ChatGPT использовался для подготовки тестовых данных и скриптов проверки, анализа результатов запусков и поиска граничных случаев.

ИИ также помог составить отчёт на основе отчётов по ИДЗ № 1 и № 2: описать многопроцессную модель, взаимодействие процессов, используемые средства IPC, интерфейс программы, способы сборки, запуска и завершения, а также результаты тестирования. Дополнительно были получены пояснения по работе с Git, созданию отдельной ветки от master и подготовке Pull Request.

Запросы:

Тестирование:

Подготовь тесты и скрипт проверки многопроцессной программы для ИДЗ № 3. Используй тесты из предыдущих заданий. Проверь все четыре режима взаимодействия процессов, FIFO, маршруты покупателей, параллельное обслуживание, закрытие магазина, максимальное число покупателей, неверные входные данные и корректную работу синхронизации.

Анализ результатов:

Проанализируй результаты тестирования многопроцессной программы. Проверь отсутствие зависаний, правильность очередей и взаимодействие процессов через разделяемую память, семафоры и очереди сообщений. Отдельно проверь завершение по SIGINT и SIGTERM, освобождение IPC-ресурсов и сохранение итогового журнала.

Подготовка отчёта:

Составь отчёт по ИДЗ № 3 на основе предыдущих отчётов, требований задания, исходного кода и результатов тестирования. Сохрани структуру и стиль прошлых отчётов. Опиши четыре режима взаимодействия процессов, используемые средства IPC, сборку, запуск, завершение и результаты проверок. Пиши кратко и не добавляй неподтверждённые сведения.

Работа с GitHub через PowerShell:

Дай команды PowerShell для создания отдельной ветки hw3 от master в репозитории acos, добавления решения в папку hw3 и создания Pull Request из hw3 в master. Сохрани предыдущие задания без изменений. Объясни, как проверить файлы перед коммитом.